

01_Limpieza_Datos_Pandas_Polars

February 2, 2026

1 Taller Práctico: Limpieza de Datos con Pandas y Polars

1.1 Objetivo

En este taller, aprenderás a realizar tareas fundamentales de limpieza de datos utilizando dos de las librerías más potentes en Python: **Pandas** y **Polars**. El objetivo es que entiendas las diferencias y similitudes entre ambas, y cómo manejar datasets del mundo real que suelen estar “sucios” (valores faltantes, tipos de datos incorrectos, duplicados, etc.).

1.2 1. Selección del Dataset

Para realizar esta práctica, necesitarás descargar un dataset de Kaggle. Te propongo las siguientes opciones (elige una):

1. **Used Cars Dataset:** Contiene información sobre coches de segunda mano. Es ideal porque suele tener columnas con texto “sucio” (e.g., precios con símbolos de moneda, kilometraje con unidades) y valores nulos.
 - *Búsqueda en Kaggle:* “Used Cars Dataset” (ej. el de Austin Reese).
2. **Netflix Movies and TV Shows:** Contiene metadatos de contenido de Netflix. Perfecto para practicar limpieza de cadenas de texto (directores, actores) y manejo de fechas.
 - *Búsqueda en Kaggle:* “Netflix Movies and TV Shows”.
3. **San Francisco Building Permits:** Un dataset más grande y “desordenado”, con muchos valores nulos y tipos de datos mezclados.
 - *Búsqueda en Kaggle:* “San Francisco Building Permits”.

Instrucciones: 1. Descarga el archivo CSV de tu elección desde Kaggle. 2. Colócalo en la misma carpeta que este notebook (o en una subcarpeta `data/`). 3. Renómbralo a algo sencillo, por ejemplo: `dataset.csv`.

1.3 2. Instalación e Importación de Librerías

Asegúrate de tener instaladas las librerías necesarias. Descomenta la siguiente línea si necesitas instalarlas.

```
[1]: # !pip install pandas polars matplotlib seaborn
```

```
[2]: import pandas as pd
import polars as pl
import numpy as np
import time # Para medir tiempos de ejecución
```

1.4 3. Carga de Datos

1.4.1 3.1 Cargar con Pandas

Carga el dataset utilizando Pandas y muestra las primeras 5 filas.

```
[3]: # Tu código aquí (Pandas)
start = time.time() # Tiempo actual previo a cargar el csv
df_pandas = pd.read_csv('dataset.csv') # Ajusta el nombre del archivo
print(f"Tiempo Pandas: {time.time()-start:.2f}s") # Muestro cuanto tardo en
    ↪cargar
df_pandas.head()
```

Tiempo Pandas: 53.47s

```
[3]:      id      url \
0  7222695916  https://prescott.craigslist.org/cto/d/prescott...
1  7218891961  https://fayar.craigslist.org/ctd/d/bentonville...
2  7221797935  https://keys.craigslist.org/cto/d/summerland-k...
3  7222270760  https://worcester.craigslist.org/cto/d/west-br...
4  7210384030  https://greensboro.craigslist.org/cto/d/trinit...

      region      region_url  price  year \
0      prescott  https://prescott.craigslist.org    6000   NaN
1  fayetteville  https://fayar.craigslist.org   11900   NaN
2  florida keys  https://keys.craigslist.org   21000   NaN
3 worcester / central MA  https://worcester.craigslist.org    1500   NaN
4      greensboro  https://greensboro.craigslist.org    4900   NaN

  manufacturer model condition cylinders  ... size  type paint_color \
0          NaN   NaN      NaN      NaN  ...  NaN   NaN      NaN
1          NaN   NaN      NaN      NaN  ...  NaN   NaN      NaN
2          NaN   NaN      NaN      NaN  ...  NaN   NaN      NaN
3          NaN   NaN      NaN      NaN  ...  NaN   NaN      NaN
4          NaN   NaN      NaN      NaN  ...  NaN   NaN      NaN

  image_url description county state lat long posting_date
0      NaN      NaN   NaN   az  NaN  NaN      NaN
1      NaN      NaN   NaN   ar  NaN  NaN      NaN
2      NaN      NaN   NaN   fl  NaN  NaN      NaN
3      NaN      NaN   NaN   ma  NaN  NaN      NaN
4      NaN      NaN   NaN   nc  NaN  NaN      NaN
```

[5 rows x 26 columns]

1.4.2 3.2 Cargar con Polars

Realiza la misma operación utilizando Polars.

```
[4]: # Tu código aquí (Polars)
start = time.time()
df_polars = pl.read_csv('dataset.csv') # Ajusta el nombre del archivo
print(f"Tiempo Polars: {time.time()-start:.2f}s")
df_polars.head()
```

Tiempo Polars: 14.08s

[4]: shape: (5, 26)

id	url	region	region_url	...	state	lat
long	posting_date					
---	---	---	---		---	---
---	---					
i64	str	str	str		str	f64
f64	str					
7222695916	https://pres	prescott	https://pres	...	az	null
null null	cott.craigslist		cott.craigslist			
	ist.or...		ist.or...			
7218891961	https://faya	fayetteville	https://faya	...	ar	null
null null	r.craigslist		r.craigslist			
	.org/c...		.org			
7221797935	https://keys	florida keys	https://keys	...	fl	null
null null	.craigslist.		.craigslist.			
	org/ct...		org			
7222270760	https://worc	worcester /	https://worc	...	ma	null
null null	ester.craigs	central MA	ester.craigs			
	list.o...		list.o...			
7210384030	https://gree	greensboro	https://gree	...	nc	null
null null	nsboro.craig		nsboro.craig			

slist....

slist....

1.5 4. Inspección General del Dataset

Antes de limpiar, debemos conocer los datos. **Tarea:** Muestra la información general (tipos de datos, columnas) y estadísticas descriptivas básicas.

```
[5]: # Pandas: info() y describe()
print("--- Pandas Info ---")
df_pandas.info()
print("\n--- Pandas Describe ---")
df_pandas.describe()
```

```
--- Pandas Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 426880 entries, 0 to 426879
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    426880 non-null  int64
1   url                   426880 non-null  object
2   region                426880 non-null  object
3   region_url            426880 non-null  object
4   price                 426880 non-null  int64
5   year                  425675 non-null  float64
6   manufacturer          409234 non-null  object
7   model                 421603 non-null  object
8   condition             252776 non-null  object
9   cylinders             249202 non-null  object
10  fuel                  423867 non-null  object
11  odometer              422480 non-null  float64
12  title_status          418638 non-null  object
13  transmission          424324 non-null  object
14  VIN                   265838 non-null  object
15  drive                 296313 non-null  object
16  size                  120519 non-null  object
17  type                  334022 non-null  object
18  paint_color           296677 non-null  object
19  image_url             426812 non-null  object
20  description            426810 non-null  object
21  county                0 non-null       float64
22  state                 426880 non-null  object
23  lat                   420331 non-null  float64
24  long                  420331 non-null  float64
25  posting_date          426812 non-null  object
```

```
dtypes: float64(5), int64(2), object(19)
memory usage: 84.7+ MB
```

```
--- Pandas Describe ---
```

```
[5]:
```

	id	price	year	odometer	county \
count	4.268800e+05	4.268800e+05	425675.000000	4.224800e+05	0.0
mean	7.311487e+09	7.519903e+04	2011.235191	9.804333e+04	NaN
std	4.473170e+06	1.218228e+07	9.452120	2.138815e+05	NaN
min	7.207408e+09	0.000000e+00	1900.000000	0.000000e+00	NaN
25%	7.308143e+09	5.900000e+03	2008.000000	3.770400e+04	NaN
50%	7.312621e+09	1.395000e+04	2013.000000	8.554800e+04	NaN
75%	7.315254e+09	2.648575e+04	2017.000000	1.335425e+05	NaN
max	7.317101e+09	3.736929e+09	2022.000000	1.000000e+07	NaN

	lat	long
count	420331.000000	420331.000000
mean	38.493940	-94.748599
std	5.841533	18.365462
min	-84.122245	-159.827728
25%	34.601900	-111.939847
50%	39.150100	-88.432600
75%	42.398900	-80.832039
max	82.390818	173.885502

```
[6]: # Polars: describe() y glimpse() (si prefieres)
print("--- Polars Describe ---")
df_polars.describe()
```

```
--- Polars Describe ---
```

```
[6]: shape: (9, 27)
```

statistic	id	url	region	...	state	lat	
long	posting_d						
---	---	---	---		---	---	---
ate							
str	f64	str	str		str	f64	f64

str							
count	426880.0	426880	426880	...	426880	420331.0	
420331.0	426812						
null_count	0.0	0	0	...	0	6549.0	
6549.0	68						

mean	7.3115e9	null	null	...	null	38.49394	
-94.74859	null						9
std	4.4732e6	null	null	...	null	5.841533	
18.365462	null						
min	7.2074e9	https://ab	SF bay	...	ak	-84.12224	
-159.8277	2021-04-0	ilene.crai	area			5	28
4T00:10:4		gslist.org					
0-0700		...					
25%	7.3081e9	null	null	...	null	34.6019	
-111.9398	null						47
50%	7.3126e9	null	null	...	null	39.1501	
-88.4326	null						
75%	7.3153e9	null	null	...	null	42.3989	
-80.83203	null						9
max	7.3171e9	https://za	zanesville	...	wy	82.390818	
173.88550	2021-05-0	nesville.c	/				2
4T23:59:4		raigslist.	cambridge				
5-0400		...					

1.6 5. Manejo de Valores Faltantes (Nulls/NaNs)

Los datos reales raramente están completos. **Tarea:** 1. Cuenta cuántos valores nulos hay por columna. 2. Decide una estrategia para una columna importante con nulos (e.g., rellenar con la media, moda, o un valor fijo como “Desconocido”, o eliminar las filas). 3. Aplica esa estrategia.

```
[7]: # --- PANDAS ---
# 1. Contar nulos
print("Nulos por columna (top 10):")
nulos = df_pandas.isnull().sum().sort_values(ascending=False).head(10)
print(nulos)
# 2. Rellenar o eliminar (Ejemplo: rellenar columna 'X' con la media)
```

```

# df_pandas['columna'] = df_pandas['columna'].fillna(valor)
df_pd = df_pandas.copy()
# Elimino columnas con muchos nulos (county 100%, size 71%)
df_pd = df_pd.drop(columns=['county', 'size'])
# Relleno categoricas
cols_cat = ['manufacturer', 'model', 'condition', 'cylinders', 'fuel', 'title_status', 'transmission', 'drive', 'type', 'paint_color']
for col in cols_cat:
    df_pd[col] = df_pd[col].fillna('unknown')
# Relleno odometer con mediana
df_pd['odometer'] = df_pd['odometer'].fillna(df_pd['odometer'].median())
# Elimino filas sin year
df_pd = df_pd.dropna(subset=['year'])
print(f"\nFilas despues de tratar nulos: {len(df_pd)}")

```

Nulos por columna (top 10):

```

county      426880
size        306361
cylinders    177678
condition    174104
VIN          161042
drive        130567
paint_color  130203
type         92858
manufacturer 17646
title_status 8242
dtype: int64

```

Filas despues de tratar nulos: 425675

```

[8]: # --- POLARS ---
# 1. Contar nulos (null_count)
print("Nulos por columna:")
print(df_polars.null_count())
# 2. Rellenar o eliminar (fill_null, drop_nulls)
# df_polars = df_polars.with_columns(pl.col('columna').fill_null(valor))
df_pl = df_polars.drop(['county', 'size'])
# Relleno categoricas con 'unknown'
cols_cat = ['manufacturer', 'model', 'condition', 'cylinders', 'fuel', 'title_status', 'transmission', 'drive', 'type', 'paint_color']
for col in cols_cat:
    df_pl = df_pl.with_columns(pl.col(col).fill_null('unknown'))
# Relleno odometer con mediana
med = df_pl.select(pl.col('odometer').median()).item()
df_pl = df_pl.with_columns(pl.col('odometer').fill_null(med))
# Elimino filas sin year
df_pl = df_pl.drop_nulls(subset=['year'])

```

```
print(f"\nFilas despues de tratar nulos: {len(df_pl)}")
```

Nulos por columna:

shape: (1, 26)

id	url	region	region_url	...	state	lat	long	posting_date
---	---	---	---		---	---	---	---
u32	u32	u32	u32		u32	u32	u32	u32
0	0	0	0	...	0	6549	6549	68

Filas despues de tratar nulos: 425675

1.7 6. Manejo de Duplicados

Tarea: Verifica si existen filas totalmente duplicadas y elimínalas.

```
[9]: # --- PANDAS ---
# Verificar y eliminar duplicados
dupes = df_pd.duplicated().sum()
print(f"Duplicados encontrados: {dupes}")
df_pd = df_pd.drop_duplicates(subset=['id'], keep='first')
print(f"Filas despues de eliminar duplicados: {len(df_pd)}")
```

Duplicados encontrados: 0

Filas despues de eliminar duplicados: 425675

```
[10]: # --- POLARS ---
# Verificar y eliminar duplicados (unique)
dupes = len(df_pl) - df_pl.unique().shape[0]
print(f"Duplicados encontrados: {dupes}")
df_pl = df_pl.unique(subset=['id'], keep='first')
print(f"Filas despues de eliminar duplicados: {len(df_pl)}")
```

Duplicados encontrados: 0

Filas despues de eliminar duplicados: 425675

1.8 7. Transformación de Datos y Limpieza de Strings

A menudo, los números vienen como texto (e.g., “\$1,000”, “50 kg”) o las fechas no tienen el formato correcto.

Tarea (Ejemplo Genérico): 1. Selecciona una columna de tipo string que debería ser numérica o fecha. 2. Limpia el string (quita símbolos como ‘\$’, ‘kg’, ‘,’). 3. Convierte la columna al tipo correcto (int, float, o datetime).

Nota: Si elegiste el dataset de coches, la columna de ‘precio’ o ‘kilometraje’ es ideal para esto.

```
[11]: # --- PANDAS ---
# Ejemplo: df_pandas['precio'] = df_pandas['precio'].str.replace('$', '').
#         ↳ astype(float)
# En este dataset de coches, 'price' y 'odometer' ya vienen como numeros, no
#         ↳ como strings tipo "$1,000". Por eso no hace falta limpiar simbolos.
# Convierto year a int y posting_date a datetime
df_pd['year'] = df_pd['year'].astype(int)
df_pd['posting_date'] = pd.to_datetime(df_pd['posting_date'], errors='coerce')
# Normalizo texto (minusculas)
for col in cols_cat:
    df_pd[col] = df_pd[col].str.lower().str.strip()
# Filtro precios raros (0 o muy altos)
df_pd = df_pd[(df_pd['price'] > 0) & (df_pd['price'] <= 500000)]
df_pd = df_pd[(df_pd['odometer'] >= 0) & (df_pd['odometer'] <= 1000000)]
print(f"Filas finales Pandas: {len(df_pd)}")
print(df_pd.dtypes)
```

/tmp/ipykernel_11453/1871789461.py:6: FutureWarning: In a future version of pandas, parsing datetimes with mixed time zones will raise a warning unless `utc=True`. Please specify `utc=True` to opt in to the new behaviour and silence this warning. To create a `Series` with mixed offsets and `object` dtype, please use `apply` and `datetime.datetime.strptime`

```
df_pd['posting_date'] = pd.to_datetime(df_pd['posting_date'], errors='coerce')
```

Filas finales Pandas: 392282

id	int64
url	object
region	object
region_url	object
price	int64
year	int64
manufacturer	object
model	object
condition	object
cylinders	object
fuel	object
odometer	float64
title_status	object
transmission	object
VIN	object
drive	object
type	object
paint_color	object
image_url	object
description	object
state	object
lat	float64
long	float64

```
posting_date      object
dtype: object
```

```
[12]: # --- POLARS ---
# Ejemplo:
# df_polars = df_polars.with_columns(
#     pl.col('precio').str.replace('$', '').cast(pl.Float64)
# )
# Igual que en Pandas, 'price' y 'odometer' ya son numericos en este dataset,
# ↪no hace falta limpiar simbolos como '$'. Solo convierto tipos y filtro.
df_pl = df_pl.with_columns([
    pl.col('year').cast(pl.Int32),
    # posting_date viene como string ISO, lo convierto a datetime
    pl.col('posting_date').cast(pl.String).str.to_datetime(format='%Y-%m-%dT%H:
    ↪M:%S%z', strict=False),
    pl.col('price').cast(pl.Float64),
    pl.col('odometer').cast(pl.Float64)
])
# Normalizo texto a minusculas y quito espacios
for col in cols_cat:
    df_pl = df_pl.with_columns(pl.col(col).str.to_lowercase().str.strip_chars())
# Filtro valores raros: precio entre 1 y 500000, odometro hasta 1 millon
df_pl = df_pl.filter(
    (pl.col('price') > 0) & (pl.col('price') <= 500000) &
    (pl.col('odometer') >= 0) & (pl.col('odometer') <= 1000000)
)
print(f"Filas finales Polars: {len(df_pl)}")
print(df_pl.schema)
```

Filas finales Polars: 392282

```
Schema([('id', Int64), ('url', String), ('region', String), ('region_url',
String), ('price', Float64), ('year', Int32), ('manufacturer', String),
('model', String), ('condition', String), ('cylinders', String), ('fuel',
String), ('odometer', Float64), ('title_status', String), ('transmission',
String), ('VIN', String), ('drive', String), ('type', String), ('paint_color',
String), ('image_url', String), ('description', String), ('state', String),
('lat', Float64), ('long', Float64), ('posting_date', Datetime(time_unit='us',
time_zone='UTC'))])
```

1.9 8. Conclusiones y Comparativa

Escribe brevemente tus conclusiones: - ¿Qué librería te pareció más intuitiva? - ¿Notaste alguna diferencia de velocidad (si el dataset era grande)? - ¿Qué dificultades encontraste en la limpieza?

¿Qué librería me pareció más intuitiva? Pandas, porque ya lo había usado antes y la sintaxis me resulta más natural. Polars con su `pl.col()` me costó al principio pero una vez le coges el truco es bastante claro.

¿Noté diferencia de velocidad? Sí, bastante. Mi ordenador no es muy potente y Pandas tardó

53,47 segundos en cargar el CSV mientras que Polars sólo 14,08 segundos. Cuatro veces más rápido Polars que Panda.

¿Qué dificultades encontré en la limpieza? Principalmente los nulos. La columna 'county' estaba 100% vacía y 'size' un 71%, así que las eliminé. Para las categóricas usé 'unknown' para no perder tantas filas. También el formato de fecha de Polars me dio problemas al principio.

1.10 9. Guardar el Dataset Limpio

Finalmente, guarda el dataset limpio en un nuevo archivo CSV.

```
[13]: # Pandas
df_pandas.to_csv('dataset_limpio_pandas.csv', index=False)

# Polars
df_polars.write_csv('dataset_limpio_polars.csv')
```

```
[ ]:
```